Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №8

з дисципліни „ Професійний Python”

на тему „ Мережеве та асинхронне програмування. REST API.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Мета роботи

Ознайомитися з мережевим програмуванням у Python, протоколом HTTP та архітектурним стилем REST. Розробити REST API на FastAPI для обліку спортивних результатів, описати й перевірити структури даних за допомогою Pydantic, організувати асинхронні HTTP-запити засобами HTTPX та asyncio, а також підготувати автоматизовані тести API.

2. Індивідуальний варіант і постановка задачі

Варіант 9 - система обліку спортивних результатів. Предметна область охоплює види спорту, спортсменів і результати змагань. Необхідно створити endpoints для перегляду й додавання спортсменів і результатів, редагування та видалення результату, перегляду результатів і рейтингу для вибраного виду спорту. Додатково потрібно обчислювати найкращий і середній результат, реалізувати асинхронне отримання даних із зовнішнього API з timeout, retry та обмеженням паралельності, а також підготувати тести.

3. Предметна область та архітектура

Дані організовано у три сутності. Sport описує вид спорту; Athlete містить ім’я, категорію та зовнішній ключ на вид спорту; Result зберігає спортсмена, числове значення, назву змагання та його дату. Зв’язки між таблицями забезпечують цілісність даних і каскадне видалення залежних записів.

HTTP-клієнт → FastAPI endpoints → Pydantic schemas → SQLAlchemy ORM → SQLite
↘ HTTPX та зовнішній API

Маршрутизатори реалізовані у sports_api/api.py. Моделі бази даних містяться у models.py, валідаційні та response-схеми - у schemas.py, а engine і сесії SQLAlchemy - у database.py. Файл external_api.py містить асинхронні запити. Такий поділ відокремлює HTTP-рівень від опису моделей і зовнішньої мережевої взаємодії.

4. Структура бази даних

Таблиця	Основні поля та обмеження
sports	id - первинний ключ; name - унікальна назва; discipline - дисципліна.
athletes	id - первинний ключ; name; category; sport_id - зовнішній ключ на sports; унікальність пари sport_id та name.
results	id - первинний ключ; athlete_id - зовнішній ключ на athletes; value - невід’ємне числове значення; competition_name; competition_date.

Для зберігання використано SQLite та SQLAlchemy 2.0. З'єднання активує перевірку зовнішніх ключів SQLite. За замовчуванням застосунок створює sports_results.db у поточній директорії. Змінна середовища SPORTS_DB_URL дає змогу вказати інший файл бази даних.

5. REST API

Метод і маршрут	**Призначення**
POST /sports	Створення виду спорту; відповідь 201, конфлікт назви - 409.
GET /sports	Перелік видів спорту.
POST /athletes	Додавання спортсмена з перевіркою sport_id.
GET /athletes	Перелік спортсменів; фільтри sport_id і name.
POST /results	Додавання результату з перевіркою athlete_id.
PATCH /results/{result_id}	Часткове оновлення результату.
DELETE /results/{result_id}	Видалення результату; відповідь 204.
GET /sports/{sport_id}/results	Результати спортсменів вибраного виду спорту.
GET /sports/{sport_id}/ranking	Рейтинг за значенням результату.
GET /sports/{sport_id}/statistics	Кількість, середнє та найкраще значення результату.

Для відсутніх ресурсів повертається HTTP 404. Повторна назва виду спорту або спортсмена в межах виду спорту обробляється як конфлікт 409. Некоректні значення запиту відхиляються FastAPI та Pydantic зі статусом 422. Створення ресурсів повертає 201 Created.

6. Pydantic-схеми та перевірка даних

Схеми SportCreate, AthleteCreate та ResultCreate перевіряють вхідні поля. Для імені й назви змагання задано довжину, ідентифікатори мають бути додатними, а значення результату не може бути від'ємним. ResultPatch допускає часткову зміну полів. Response-схеми обмежують і документують структуру відповідей API. FastAPI використовує ці схеми для автоматичного формування OpenAPI та Swagger UI.

POST /results
{ "athlete_id": 1, "value": 52.4,
"competition_name": "City Cup", "competition_date": "2026-10-01" }

7. Асинхронні запити

У external_api.py використано httpx.AsyncClient із timeout 5 секунд. Функція get_json_with_retry повторює запит до трьох разів після мережевої помилки або помилки HTTP-статусу, роблячи паузи між спробами. fetch_many_public_posts створює asyncio.Task для кількох запитів, збирає результати через asyncio.gather і обмежує

кількість одночасних запитів семафором до трьох. Endpoint /async/public-posts обгортає виконання загальним timeout і повертає 504 при його перевищенні.

Для демонстрації використовується тестовий сервіс JSONPlaceholder. Це приклад асинхронної мережевої взаємодії, а не зовнішнє джерело спортивної статистики. Його робота потребує доступу до Інтернету.

8. Автоматизовані тести

У tests/conftest.py налаштовано ізольовану SQLite-базу в пам'яті та перевизначення залежності сесії. У tests/test_api.py передбачено чотири сценарії: створення й перегляд видів спорту та спортсменів; створення, оновлення та видалення результату; перевірка помилкового значення і відповіді 404; формування рейтингу та статистики. Наявність тестів описує підготовлене покриття сценаріїв; фактичне виконання тестового набору для цього звіту не проводилося.

9. Запуск проєкту

Для локального запуску у Windows PowerShell у корені проєкту потрібно створити віртуальне середовище, встановити проєкт разом із тестовими залежностями та запустити Uvicorn:

```
python -m venv .venv
 .\.venv\Scripts\Activate.ps1
 python -m pip install -e ".[test]"
 python -m uvicorn sports_api.api:app --reload
```

Після запуску документація доступна за адресою http://127.0.0.1:8000/docs. Тести можна запустити окремо командою python -m pytest -v.

10. Висновки

У лабораторній роботі розглянуто принципи HTTP та REST і створено FastAPI-застосунок для обліку спортивних результатів. Реалізовано моделі видів спорту, спортсменів і результатів, перевірку вхідних даних, основні endpoints, рейтинг і агреговану статистику. Асинхронний модуль демонструє HTTPX, asyncio.Task, gather, timeout, retry та Semaphore. Для API підготовлено ізольовані автоматизовані тести.

Перед використанням наявної бази з лабораторної №7 слід налаштувати SPORTS_DB_URL і перевірити відповідність її схеми моделям цього застосунку. README проєкту описує таке підключення, але стандартний запуск використовує окремий файл SQLite. Це слід врахувати під час інтеграції з попередньою лабораторною роботою.